__/__/ 36

Q) Write a C program to simulate
a) Producer-Consumer problem using Semaphore
b) Dining-Philosopher's problem

A)
```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER 3
#define N 4

/* Producer - Consumer */

int buffer[BUFFER];
int in=0; out= 0;

sem_t empty, full, mutex;

void* producer(void *arg){
    for(int i=0; i<5; i++){
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
```

```
        printf("Produced: %d at buffer[%d]\n", i, in);
        in = (in+1) % BUFFER;
        sem_post(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}

void * consumer(void *arg){
    for(int i=0; i<5; i++){
        usleep(200000);
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out+1) % BUFFER
        sem_post(&mutex);
        sem_post(&empty);
    }
    return NULL;
}
```

```
void run_p_c (){
    pthread_t p, c;
    sem_init(& empty, 0, BUFFER);
    sem_init (& full, 0, 0);
    sem_init( & mutex, 0, 1);
    pthread_create (&p, NULL, producer, NULL);
    pthread_create (&c, NULL, consumer, NULL);
    pthread_join (p, NULL);
    pthread_join (c, NULL);
    sem_destroy (& empty);
    sem_destroy (& full);
    sem_destroy (& mutex);
}
```

```
/* Dining Philosopher */

sem_t  cs[N];
sem_t  room;

void* philosopher( void *num){
    int id = *(int *) num;
    printf("Philosopher %d is thinking.\n", id);
    sleep (1);
    sem_wait (& room);
```

```
printf("Philosopher %d picked up left fork %d.\n", id, id);
sem_wait(&cs[id]);

printf("Philosopher %d picked up right fork %d.\n", id,
    (id+1) % N);
sem_wait(&cs[(id+1) % N]);

printf("Philosopher %d is eating.\n", id);
sleep(1);

printf("Philosopher %d put down forks %d & %d.\n",
    id, id, (id+1) % N);

sem_post(&cs[id]);
sem_post(&cs[(id+1) % N]);

sem_post(&room);

return NULL;
}

void run_d_p() {
  pthread_t ph[N];
  int ids[N];
  sem_init(&room, 0, N-1);
```

```
for (int i=0; i<N; i++)
    sem_init(&cs[i], 0, 1);

for(int i=0; i<N; i++) {
    ids[i] = i;
    pthread_create(&ph[i], NULL, philosopher, &ids[i]);
    usleep(150000);
}

for (int i=0; i<N; i++)
    pthread_join(ph[i], NULL);

for (int i=0; i<N; i++)
    sem_destroy(&cs[i]);

sem_destroy(&room);
}

int main(){
  int choice;
  while (1){
    printf("1. PC \n 2. DP \n 3. Exit \n");
    printf("Enter choice: ");
    scanf("%d", &choice);
```

```
switch (choice) {
        case 1 :  run_p_d();
                  break;

        case 2 :  run_d_p();
                  break;
        case 3 :  exit(0);
        default : printf("Invalid");
    }
}
return 0;
}
```

21/4/26

Traing :

| Action | Buffer | Mutex | empty | full |
|---|---|---|---|---|
| P1 | 1 - - | 1 | 2 | 1 |
| c1 | - - - | 1 | 3 | 0 |
| P2 | - 2 - | 1 | 2 | 1 |
| c2 | - - - | 1 | 3 | 0 |
| P3 | - - 3 | 1 | 2 | 1 |
| P4 | 4 - 3 | 1 | 1 | 2 |
| c3 | 4 - - | 1 | 2 | 1 |

8) Write a C program to simulate Banker's Algo.
Safety Algorithm

```
#include <stdio.h>
#include <stdbool.h>

int main(){
    int n, m;
    printf("No. of processes: ");
    scanf("%d", &n);
    printf("No. of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int available[m];

    printf("Enter allocation matrix: \n");
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            scanf("%d", &alloc[i][j]);
        }
    }
```

```
printf("Enter Maximum Demand matrix: \n");
for(int i=0; i<n; i++){
    for(int j=0; j<m; j++){
        scanf("%d", &max[i][j]);
    }
}

printf("Enter Available Resources: \n");
for(int i=0; i<m; i++){
    scanf("%d", &available[i]);
}
```

```
for(int i=0; i<n; i++){
    for(int j=0; j<m; j++){
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int work[m];
bool finish[n];
int safeSeq[n];

for(int i=0; i<m; i++)
    work[i] = available[i];
```

__/__/44

```
for (int i=0; i<n; i++)
    finish [i] = false;

int count =0;

while (count < n){
    bool found = false;
    for( int i=0; i<n; i++) {
        if (!finish [i]) {
            bool canExecute = true;

            for(int j=0; j<m; j++){
                if (need [i][j] > work[j]) {
                    canExecute = false;
                    break;
                }
            }
            if (canExecute){
                for (int j=0; j<m; j++)
                    work [j] += alloc [i][j];
                safeSeq [count++] = i;
                finish [i] = true;
                found = true;
            }
        }
    }
}
```

```
        if (!found){
            printf(" UNSAFE\n");
            return 0;
        }
    }

    printf("\nSystem is in safe state.\n");
    printf(" Safe sequence is : ");

    for(int i=0; i<n; i++){
        printf (" P%d ", safeSeq[i]);
        if ( i != n-1)
            printf(" -> ");
    }
    printf ("\n");
    return 0;
}
```

28/4/26

o/p: No. of process: 5

No. of resource: 4

Enter Allocation Matrix:

0 0 1 2

1 0 0 0

1 3 5 4

0 6 3 2

0 0 1 4

Enter Maximum Demand Matrix

0 0 1 2

1 7 5 0

2 3 5 6

0 6 5 2

0 6 5 6

Enter Available Resources:

1 5 2 0

Safe state

seq: $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

__/__/46

→ Resource Request Algorithm :

```
#include <stdio.h>
#include <stdbool.h>

bool safetyAlgo(int n, int m, int alloc[n][m],
                int need[n][m], int available[m]) {
    int work[m];
    bool finish[n];
    int safeSeq[n];

    for(int i=0; i<m; i++)
        work[i] = available[i];
    for(int i=0; i<n; i++)
        finish[i] = false;

    int count = 0;
    while (count < n) {
        bool found = false;
        for(int i=0; i<n; i++) {
            if(!finish[i]) {
                bool canExec = true;
                for(int j=0; j<m; j++) {
                    if(need[i][j] > work[j]){
                        canExec = false;
                        break;
                    }
                }
                if(canExec) {
                    for(int k=0; k<m; k++)
                        work[k] += alloc[i][k];
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }
        if(!found) {
            printf("System is not in safe state\n");
            return false;
        }
    }
    printf("System is in safe state.\nSafe sequence is: ");
    for(int i=0; i<n; i++)
        printf("%d ", safeSeq[i]);
    printf("\n");
    return true;
}

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int alloc[n][m], need[n][m], available[m];
    printf("Enter allocation matrix:\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter need matrix:\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            scanf("%d", &need[i][j]);
    printf("Enter available resources:\n");
    for(int j=0; j<m; j++)
        scanf("%d", &available[j]);
    safetyAlgo(n, m, alloc, need, available);
    return 0;
}
```

```
                if (canExec){
                    for(int j = 0; j<m; j++)
                        work[j] += alloc[i][j];
                    safeSeq[count++] = i;
                    finish[i]= true;
                }
            }
        }
        if (!found){
            return false;
        }
    }
    printf("\nSystem is in a safe state. \n Safe Sequence
            is: ");
    for(int i=0; i<n; i++){
        printf(" P%d", safeSeq[i]);
        if (i!=n-1) printf(" -> ");
    }
    printf("\n");
    return true;

void resourceReq(int n, int m, int alloc[n][m],
                 int need[n][m], int available[m]){
    int process;
    printf("\nEnter process no making request (0-%d): ",
            n-1);
```

```
scanf("%d", &process);

int request[m];
printf("Enter request vector: ");
for(int i=0; i<m; i++)
    scanf("%d", &request[i]);

for(int i=0; i<m; i++){
    if(request[i] > need[process][i]){
        printf("\n Error\n");
        return;
    }
}
for(int i=0; i<m; i++){
    if(request[i] > available[i]){
        printf("Resource not available, must wait\n");
        return;
    }
}
for(int i=0; i<m; i++){
    available[i] -= request[i];
    alloc[process][i] += request[i];
    need[process][i] -= request[i];
}
```

```
if (safetyAlgo(n, m, alloc, need, available)) {
    printf("\n Request can be granted \n");
} else {
    printf("\n Request can't be granted (unsafe) \n");
    for(int i=0; i<m; i++){
        available[i] += request[i];
        alloc[process][i] -= request[i];
        need[process][i] += request[i];
    }
}
```

~~int main(){~~

```
int main(){
  int n, m;
  printf("Enter no. of processes: ");
  scanf("%d", &n);
  printf("Enter no. of resources: ");
  scanf("%d", &m);
  int alloc[n][m], max[n][m], need[n][m];
  int available[m];

  printf("Enter allocation matrix: \n");
  for(int i=0; i<n; i++)
      for(int j=0; j<m; j++)
          scanf("%d", &alloc[i][j]);
```

```
printf("Enter Max Demand matrix: \n");
for(int i=0; i<n; i++)
    for(int j=0; j<m; j++)
        scanf("%d", &max[i][j]);

printf("Enter available resources: \n");
for(int i=0; i<m; i++)
    scanf("%d", &available[i]);

for(int i=0; i<n; i++)
    for(int j=0; j<m; j++)
        need[i][j] = max[i][j] - alloc[i][j];

if(!safetyAlgo(n, m, alloc, need, available)){
    printf("In System is unsafe initially \n");
    return 0;
}

resourceReq(n, m, alloc, need, available);

return 0;
}
```

o/p: Enter no. of processes: 5

Enter no. of resources: 3

Enter allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter max matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter available resources:

3 3 2

System is in a safe state

Safe sequence is: P1 → P3 → P4 → P0 → P2

Enter process no. making request (0-4):

Enter request vector: 1 0 2

Request can be granted

Safe Sequence: P1 → P3 → P4 → P0 → P2

5/5/26

=) Deadlock detection

```
#include <stdio.h>
#define MAX 10

int main() {
    int n, m;
    int alloc[MAX][MAX], req[MAX][MAX];
    int available[MAX], work[MAX], finish[MAX];
    int i, j, k;

    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter no. of resources: ");
    scanf("%d", &m);
    printf("\nEnter Allocation matrix:\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);

    printf("\nEnter Request Matrix:\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            scanf("%d", &req[i][j]);
```

```
printf("Enter available resources: \n");
for (int j=0; j<m; j++)
    scanf("%d", &available[j]);

for (j=0; j<m; j++)
    work[j] = available[j];

for (j=0; i<n; j++)
    finish[i] = 0;

int found;
do {
    found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int canExec = 1;

            for (j=0; j<m; j++) {
                if (req[i][j] > work[j]) {
                    canExec = 0;
                    break;
                }
            }
            if (canExec) {
                for(k=0; k<m; k++)
                    work[k] += alloc[i][k];
```

```
                finish[i] = 1;
                found = 1;
            }
        }
    }
} while(found);

int deadlock = 0;
printf("\n Deadlocked Processes: ");
for (i=0; i<n; i++){
    if (finish[i] == 0){
        printf("P%d ", i);
        deadlock = 1;
    }
}
if (deadlock == 0)
    printf(" NO deadlock \n");
else
    printf("\nSystem is in deadlock!\n");

return 0;
}
```

O/p: Enter no. of processes: 4

Enter no. of resources: 3

Enter Allocation matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request matrix:

0 0 1

1 0 2

0 0 0

3 3 0

Enter available resources:

0 0 0

Deadlocked processes: P3

System is in deadlock!

5/5/26

__/__/56

7) Write a C program to simulate the following contiguous memory allocation techniques

a) First fit

b) Best fit

c) Worst fit

```
#include <stdio.h>

#define MAX 10

void firstFit (int blockSize[], int blocks, int pSize[], int p){
    int alloc [MAX];
    for(int i = 0; i<p ; i++)  alloc [i]= -1 ;

    for (int i = 0;  i<p;  i++){
        for (int j= 0 ; j< blocks; j++){
            if (blockSize[j] >= pSize [i]){
                alloc [i] = j;
                blocksize [j] = 0;
                break;
            }
        }
    }

printf ("\nFirst fit Allocation: \n");
```

```
    for (int i=0; i<p; i++){
        if (alloc[i] != -1)
            printf("Process %d allocated to Block %d\n", i+1,
                    alloc[i]+1);
        else
            printf("Process %d not allocated\n", i+1);
    }
}

void bestfit(int blocksize[], int blocks, int psize[], int p){
    int alloc[MAX];
    for(int i=0; i<p; i++) alloc[i] = -1;

    for(int i=0; i<p; i++){
        int bestIdx = -1;
        for(int j=0; j<blocks; j++){
            if(blocksize[j] >= psize[i]){
                if (bestIdx == -1 || blocksize[j] < blocksize[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1){
            alloc[i] = bestIdx;
            blocksize[bestIdx] = 0;
        }
    }
}
```

```
printf("\n Best Fit Allocation:\n");
for(int i = 0; i<p; i++){
    if(alloc[i] != -1)
        printf("Process %d allocated to block %d\n", i+1,
                alloc[i] + 1);
    else
        printf("Process %d not allocated\n", i+1);
    }
}
```

```
void worstfit(int blockSize[], int blocks, int pSize[], int p){
    int alloc[MAX];
    for(int i=0; i<p; i++) alloc[i] = -1;

    for(int i=0; i<p; i++) {
        int worstIdx = -1;
        for(int j=0; j<blocks; j++) {
            if(blockSize[j]>=pSize[i]) {
                if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if(worstIdx != -1){
            alloc[i] = worstIdx;
            blockSize[worstIdx] = -1;
        }
    }
```

```
printf("\n Worst Fit Allocation \n");
for (int i=0; i<p; i++) {
    if (alloc[i] != -1)
        printf("Process %d allocated to Block %d \n", i+1,
                alloc[i]+1);
    else
        printf("Process %d not allocated\n", i+1);
    }
}

int main() {
    int blocks, processes;
    int blockSize[MAX], pSize[MAX];

printf("Enter no of blocks: "); scanf("%d", &blocks);
printf("Enter size of blocks: ");
for (int i=0; i<block; i++) scanf("%d", &blockSize[i]);

printf("Enter no of processes: "); scanf("%d", &process);
printf("Enter size of processes: ");
for (int i=0; i<process; i++) scanf("%d", &pSize[i]);

firstFit(blockSize, blocks, pSize, processes);
bestFit(blockSize, blocks, pSize, processes);
worstFit(blockSize, blocks, pSize, processes);
```

```
return 0;
}
```

o/p: Enter no of blocks: 6

Enter size of blocks: 300 600 350 200 750 125

Enter no. of processes: 5

Enter sizes of processes: 115, 500 358 200 375

First Fit allocation:

| Process | P Size | Block No. |
|---|---|---|
| 1 | 115 | 1 |
| 2 | 500 | 2 |
| 3 | 358 | 5 |
| 4 | 200 | 3 |
| 5 | 375 | Not allocated |

Best Fit Allocation:

| Process | Psize | Block No |
|---|---|---|
| 1 | 115 | 6 |
| 2 | 500 | 2 |
| 3 | 358 | 5 |
| 4 | 200 | 4 |
| 5 | 375 | Not allocated. |

__/__/61

Worst fit Allocation

| Process | Size | Block NO. |
|---|---|---|
| 1 | 115 | 5 |
| 2 | 500 | 2 |
| 3 | 358 | Not allocated |
| 4 | 200 | 3 |
| 5 | 375 | Not allocated |

12/05/26

__/__/62

*) Write a C program to simulate page replacement algorithms:

a) FIFO
b) LRU
c) Optimal

=) ~~#include <stdio.h>~~
~~#define MAX 100~~
~~#define~~

```
#include <stdio.h>
#include <string.h>
#define FRAMES 3
#define PAGES 12

int pages[PAGES] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3};

int inFrames(int frames[], int page) {
  for(int i=0; i < FRAMES; i++)
    if (frames[i] == page) return 1;
  return 0;
}
```

```
void FIFO (){
  int frames [FRAMES] = {-1, -1, -1};
  int faults = 0; ptr = 0;

  printf ("\n --- FIFO --- \n");
  for (int i=0; i < PAGES; i++) {
    if (!inFrames ( frames, pages[i])) {
      frames [ptr] = pages [i];
      ptr = (ptr +1) % FRAMES;
      faults ++;
      printf ("Page %d → [%d %d %d] FAULT \n", pages (i),
          frames (0), frames (1), frames (2));
    } else {
      printf ("Page %d → [%d %d %d] HIT \n", pages (i),
          frames [0], frames (1), frames (2));
    }
  }
  printf (" Total faults: %d \n", faults);
}

void LRU (){
  int frames [FRAMES] = {-1, -1, -1};
  int lastUsed [FRAMES] = {0};
  int faults = 0;
```

```
printf("\n---LRU---\n");
for(int i=0; i<PAGES; i++){
  if(!inFrames(frames, pages[i])){
    int replace = 0;
    for(int j=0; j<FRAMES; j++){
      if(frames[j]==-1){replace=j; break;}
      if(lastUsed[j]<lastUsed[replace]) replace=j;
    }
    frames[replace] = pages[i];
    lastUsed[replace] = i;
    faults++;
    printf("Page %d → [%d %d %d] FAULT\n", pages[i],
           frames[0], frames[1], frames[2]);
  } else {
    for(int j=0; j<FRAMES; j++)
      if(frames[j]==pages[i]) lastUsed[j]=i;
    printf("Page %d → [%d %d %d] HIT\n", pages[i],
           frames[0], frames[1], frames[2]);
  }
}
printf("Total faults: %d\n", faults);
}
```

```
void Optimal() {
  int frames[FRAMES] = {-1, -1, -1};
  int faults = 0, filled = 0;

  printf("\n --- Optimal ---\n");
  for(int i=0; i<PAGES; i++){
    if(!inFrames(frames, pages[i])) {
      if (filled < FRAMES) {
        frames[filled++] = pages[i];
      } else {
        int replace = 0; farthest = -1;
        for(int j=0; j<FRAMES; j++) {
          int nextUse = PAGES;
          for(int k=i+1; k<PAGES; k++){
            if(pages[k] == frames[j]) {nextUse=k; break;}
          }
          if(nextUse > farthest){farthest = nextUse; replace=j;}
        }
        frames[replace] = pages[i];
      }
      faults++;
      printf("Pages %d -> [%d %d %d] FAULT\n", pages[i],
          frames[0], frames[1], frames[2]);
    } else {
      printf("Pages %d -> [%d %d %d] HIT\n", pages[i],
          frames[0], frames[1], frames[2]);
    }
```

__/__/66

```
    }
    printf("Total faults : %d\n", faults);
}

int main() {
    printf("Page string : 7 0 1 2 0 3 0 4 2 3 0 3");
    printf("\n Frames : %d", FRAMES);
    FIFO();
    LRU();
    Optimal();
    return 0;
}
```

o/p Page String : 7 0 1 2 0 3 0 4 2 3 0 3

Frames : 3

~~7 → [7 -1 -1] fault~~

---FIFO---

7 → [7 -1 -1] fault

0 → [7 0 -1] fault

1 → [7 0 1] fault

2 → [2 0 1] fault

0 → [2 0 1] HIT

3 → [2 3 1] fault

0 → [2 3 0] fault

4 → [4 3 0] fault

2 → [4 2 0] fault

3 → [4 2 3] fault

0 → [0 2 3] fault

3 → [0 2 3] HIT

Total faults = 10

--- LRU ---

7 → [7 -1 -1] fault

0 → [7 0 -1] fault

1 → [7 0 1] fault

2 → [2 0 1] fault

0 → [2 0 1] HIT

3 → [2 0 3] fault

0 → [2 0 3] HIT

4 → [4 0 3] fault

2 → [4 0 2] fault

3 → [4 3 2] fault

0 → [0 3 2] fault

3 → [0 3 2] HIT

26/5/26

Total faults: 9

--- Optimal ---

7 → [7 -1 -1] fault

0 → [7 0 -1] fault

1 → [7 0 1] fault

2 → [2 0 1] fault

0 → [2 0 1] HIT

3 → [2 0 3] fault

Total faults: 7

0 → [2 0 3] HIT

4 → [2 4 3] fault

2 → [2 4 3] HIT

3 → [2 4 3] HIT

0 → [0 4 3] fault

3 → [0 4 3] Hit